Time Complexity Lab Exercises

Exercise No: 1

Merge Sort

Aim

To write a Python program to sort a list of elements using Merge Sort.

Algorithm

Step 1: Start
Step 2: Read the list of elements
Step 3: Divide the list into two halves
Step 4: Recursively sort both halves
Step 5: Merge the two sorted halves
Step 6: Display the sorted list
Step 7: Stop

Source Code

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr)//2
        L = arr[:mid]
        R = arr[mid:]

        merge_sort(L)
        merge_sort(R)

        i = j = k = 0

        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                arr[k] = L[i]
                i += 1
```

```
            else:
                arr[k] = R[j]
                j += 1
            k += 1

        while i < len(L):
            arr[k] = L[i]
            i += 1
            k += 1

        while j < len(R):
            arr[k] = R[j]
            j += 1
            k += 1

arr = list(map(int, input("Enter elements: ").split()))
merge_sort(arr)
print("Sorted list:", arr)
```

Sample Output

Enter elements: 38 27 43 3 9 82 10

Sorted list: [3, 9, 10, 27, 38, 43, 82]

Exercise No: 2

Quick Sort

Aim

To write a Python program to sort a list using Quick Sort.

Algorithm

Step 1: Start
Step 2: Read the list of elements
Step 3: Select a pivot element
Step 4: Partition the list into elements smaller and larger than pivot
Step 5: Recursively sort the partitions
Step 6: Display sorted list
Step 7: Stop

Source Code

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[len(arr)//2]
    left = [x for x in arr if x < pivot]
    middle = [x for x in arr if x == pivot]
    right = [x for x in arr if x > pivot]
    return quick_sort(left) + middle + quick_sort(right)

arr = list(map(int, input("Enter elements: ").split()))
sorted_arr = quick_sort(arr)
print("Sorted list:", sorted_arr)
```

Sample Output

Enter elements: 29 10 14 37 13

Sorted list: [10, 13, 14, 29, 37]

Exercise No: 3

Linear Search

Aim

To write a Python program to find an element in a list using Linear Search.

Algorithm

Step 1: Start
Step 2: Read the list and element to search
Step 3: Traverse the list and compare each element
Step 4: If element found, display index
Step 5: If not found, display "Element not found"
Step 6: Stop

Source Code

```
arr = list(map(int, input("Enter elements: ").split()))

x = int(input("Enter element to search: "))


found = False

for i in range(len(arr)):

    if arr[i] == x:

        print("Element found at index", i)

        found = True

        break

if not found:

    print("Element not found")
```

Sample Output

Enter elements: 5 8 12 20 7

Enter element to search: 12

Element found at index 2

Exercise No: 4

Binary Search

Aim

To write a Python program to find an element in a sorted list using Binary Search.

Algorithm

Step 1: Start
Step 2: Read sorted list and element to search
Step 3: Set low = 0 and high = n-1
Step 4: Repeat while low ≤ high
    a) mid = (low + high)//2
    b) If arr[mid] == element, display index and stop
    c) If arr[mid] < element, set low = mid + 1
    d) Else set high = mid - 1
Step 5: If element not found, display "Element not found"
Step 6: Stop

Source Code

```
arr = list(map(int, input("Enter sorted elements: ").split()))

x = int(input("Enter element to search: "))


low = 0

high = len(arr) - 1

found = False


while low <= high:

    mid = (low + high)//2

    if arr[mid] == x:

        print("Element found at index", mid)

        found = True

        break

    elif arr[mid] < x:
```

```
        low = mid + 1
    else:
        high = mid - 1

if not found:
    print("Element not found")
```

Sample Output

Enter sorted elements: 2 4 6 8 10

Enter element to search: 8

Element found at index 3

Exercise No: 5

Huffman Encoding

Aim

To write a Python program to generate Huffman codes for a given set of characters using Greedy Algorithm.

Algorithm

Step 1: Start
Step 2: Read characters and their frequencies
Step 3: Create a priority queue with nodes (character, frequency)
Step 4: Repeat until only one node remains:
    a) Extract two nodes with minimum frequency
    b) Create a new node with sum of frequencies
    c) Insert the new node back into the queue
Step 5: Traverse the Huffman tree to assign codes
Step 6: Display codes for each character
Step 7: Stop

Source Code

```
import heapq


class Node:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None
    def __lt__(self, other):
        return self.freq < other.freq


def huffman_codes(root, code="", codes={}):
    if root is None:
```

```
        return
    if root.char is not None:
        codes[root.char] = code
    huffman_codes(root.left, code + "0", codes)
    huffman_codes(root.right, code + "1", codes)
    return codes

chars = input("Enter characters: ").split()
freqs = list(map(int, input("Enter frequencies: ").split()))

heap = [Node(chars[i], freqs[i]) for i in range(len(chars))]
heapq.heapify(heap)

while len(heap) > 1:
    left = heapq.heappop(heap)
    right = heapq.heappop(heap)
    merged = Node(None, left.freq + right.freq)
    merged.left = left
    merged.right = right
    heapq.heappush(heap, merged)

root = heap[0]
codes = huffman_codes(root)
print("Huffman Codes:", codes)
```

Sample Output

Enter characters: A B C D E

Enter frequencies: 5 9 12 13 16

Huffman Codes: {'A': '1100', 'B': '1101', 'C': '100', 'D': '101', 'E': '111'}

Exercise No: 6

Kruskal's Algorithm

Aim

To write a Python program to find the Minimum Spanning Tree of a graph using Kruskal's Algorithm.

Algorithm

Step 1: Start
Step 2: Read vertices and edges
Step 3: Sort edges in ascending order of weight
Step 4: Initialize each vertex as a separate set
Step 5: For each edge, if it does not form a cycle, include it in MST
Step 6: Repeat until n-1 edges are included
Step 7: Display MST edges and total cost
Step 8: Stop

Source Code

```
def find(parent, i):
    while parent[i] != i:
        i = parent[i]
    return i


def union(parent, x, y):
    parent[x] = y


n = int(input("Enter number of vertices: "))
e = int(input("Enter number of edges: "))


edges = []
for _ in range(e):
    u, v, w = map(int, input("Enter u v weight: ").split())
    edges.append((w, u, v))
```

```
edges.sort()
parent = [i for i in range(n)]
mincost = 0

for w, u, v in edges:
    x = find(parent, u)
    y = find(parent, v)
    if x != y:
        print("Edge:", u, "-", v, "Cost:", w)
        mincost += w
        union(parent, x, y)

print("Minimum Cost =", mincost)
```

Sample Output

```
Enter number of vertices: 4
Enter number of edges: 5
Enter u v weight: 0 1 10
Enter u v weight: 0 2 6
Enter u v weight: 0 3 5
Enter u v weight: 1 3 15
Enter u v weight: 2 3 4

Edge: 2 - 3 Cost: 4
Edge: 0 - 3 Cost: 5
Edge: 0 - 1 Cost: 10
Minimum Cost = 19
```

Exercise No: 7

Prim's Algorithm

Aim

To write a Python program to find the Minimum Spanning Tree of a graph using Prim's Algorithm.

Algorithm

Step 1: Start
Step 2: Read number of vertices and adjacency matrix
Step 3: Initialize visited vertices with starting vertex
Step 4: Repeat n-1 times:
    a) Select minimum edge connecting visited and unvisited vertices
    b) Include the edge in MST
Step 5: Display edges and total minimum cost
Step 6: Stop

Source Code

```
import sys


n = int(input("Enter number of vertices: "))

cost = []


print("Enter adjacency matrix:")

for i in range(n):

   cost.append(list(map(int, input().split())))


visited = [False]*n

visited[0] = True

edges = 0

mincost = 0


while edges < n-1:
```

```
    minimum = sys.maxsize
    a = b = -1
    for i in range(n):
        if visited[i]:
            for j in range(n):
                if not visited[j] and cost[i][j] != 0:
                    if cost[i][j] < minimum:
                        minimum = cost[i][j]
                        a, b = i, j
    print("Edge:", a, "-", b, "Cost:", minimum)
    mincost += minimum
    visited[b] = True
    edges += 1

print("Minimum Cost =", mincost)
```

Sample Output

```
Enter number of vertices: 4
Enter adjacency matrix:
0 2 3 6
2 0 4 5
3 4 0 7
6 5 7 0

Edge: 0 - 1 Cost: 2
Edge: 0 - 2 Cost: 3
Edge: 1 - 3 Cost: 5
Minimum Cost = 10
```

Exercise No: 8

0/1 Knapsack Problem

Aim

To write a Python program to solve the 0/1 Knapsack problem using Dynamic Programming.

Algorithm

Step 1: Start
Step 2: Read number of items n
Step 3: Read values and weights of the items
Step 4: Read knapsack capacity W
Step 5: Create a table K[n+1][W+1] initialized to 0
Step 6: Fill the table using the formula:
    If weight[i-1] ≤ w: K[i][w] = max(value[i-1] + K[i-1][w-weight[i-1]], K[i-1][w])
    Else: K[i][w] = K[i-1][w]
Step 7: Maximum value = K[n][W]
Step 8: Display result
Step 9: Stop

Source Code

```
n = int(input("Enter number of items: "))


values = []

weights = []


print("Enter value and weight of items:")

for _ in range(n):

    v, w = map(int, input().split())

    values.append(v)

    weights.append(w)


W = int(input("Enter knapsack capacity: "))
```

```
K = [[0]*(W+1) for _ in range(n+1)]


for i in range(1, n+1):
    for w in range(W+1):
        if weights[i-1] <= w:
            K[i][w] = max(values[i-1] + K[i-1][w-weights[i-1]], K[i-1][w])
        else:
            K[i][w] = K[i-1][w]


print("Maximum value =", K[n][W])
```

Sample Output

Enter number of items: 3

Enter value and weight of items:

100 20

50 10

150 30

Enter knapsack capacity: 50

Maximum value = 250

Exercise No: 9

Longest Common Subsequence (LCS)

Aim

To write a Python program to find the Longest Common Subsequence of two strings using Dynamic Programming.

Algorithm

Step 1: Start
Step 2: Read two strings X and Y
Step 3: Create a table L of size (m+1) x (n+1), initialized to 0
Step 4: For i = 1 to m, j = 1 to n:
    a) If X[i-1] == Y[j-1], L[i][j] = L[i-1][j-1] + 1
    b) Else, L[i][j] = max(L[i-1][j], L[i][j-1])
Step 5: Display L[m][n] as length of LCS
Step 6: Stop

Source Code

```
X = input("Enter first string: ")

Y = input("Enter second string: ")

m = len(X)

n = len(Y)

L = [[0]*(n+1) for _ in range(m+1)]

for i in range(1, m+1):

    for j in range(1, n+1):

        if X[i-1] == Y[j-1]:

            L[i][j] = L[i-1][j-1] + 1

        else:

            L[i][j] = max(L[i-1][j], L[i][j-1])

print("Length of LCS =", L[m][n])
```

Sample Output

Enter first string: AGGTAB

Enter second string: GXTXAYB

Length of LCS = 4

Exercise No: 10

Floyd-Warshall Algorithm (All Pairs Shortest Path)

Aim

To write a Python program to find shortest paths between all pairs of vertices using Floyd-Warshall Algorithm.

Algorithm

Step 1: Start
Step 2: Read number of vertices n
Step 3: Read weight matrix D
Step 4: For k = 0 to n-1:
    For i = 0 to n-1:
        For j = 0 to n-1:
            D[i][j] = min(D[i][j], D[i][k] + D[k][j])
Step 5: Display the shortest path matrix
Step 6: Stop

Source Code

```
n = int(input("Enter number of vertices: "))


print("Enter weight matrix:")

dist = []

for _ in range(n):

    dist.append(list(map(int, input().split())))


for k in range(n):

    for i in range(n):

        for j in range(n):

            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])


print("Shortest path matrix:")
```

```
for row in dist:
    print(row)
```

Sample Output

```
Enter number of vertices: 3
Enter weight matrix:
0 5 999
50 0 15
30 999 0

Shortest path matrix:
[0, 5, 20]
[45, 0, 15]
[30, 35, 0]
```

Exercise No: 11

Breadth-First Search (BFS)

Aim

To write a Python program to traverse a graph using Breadth-First Search (BFS).

Algorithm

Step 1: Start
Step 2: Read number of vertices n and adjacency matrix
Step 3: Initialize a queue and visited array
Step 4: Enqueue starting vertex and mark it visited
Step 5: While queue is not empty:
    a) Dequeue vertex v
    b) Print v
    c) Enqueue all unvisited adjacent vertices and mark them visited
Step 6: Stop

Source Code

```
from collections import deque


n = int(input("Enter number of vertices: "))

graph = []


print("Enter adjacency matrix:")

for _ in range(n):

    graph.append(list(map(int, input().split())))


start = int(input("Enter starting vertex: "))

visited = [False]*n

queue = deque([start])

visited[start] = True


print("BFS Traversal:")
```

```
while queue:
    v = queue.popleft()
    print(v, end=" ")
    for i in range(n):
        if graph[v][i] and not visited[i]:
            queue.append(i)
            visited[i] = True
```

Sample Output

Enter number of vertices: 4

Enter adjacency matrix:

0 1 1 0

1 0 0 1

1 0 0 1

0 1 1 0

Enter starting vertex: 0

BFS Traversal:

0 1 2 3

Exercise No: 12

Depth-First Search (DFS)

Aim

To write a Python program to traverse a graph using Depth-First Search (DFS).

Algorithm

Step 1: Start
Step 2: Read number of vertices n and adjacency matrix
Step 3: Initialize visited array
Step 4: Define recursive DFS function:
    a) Mark vertex as visited
    b) Print vertex
    c) Call DFS recursively for unvisited adjacent vertices
Step 5: Call DFS from starting vertex
Step 6: Stop

Source Code

```
def dfs(v, visited, graph):
    visited[v] = True
    print(v, end=" ")
    for i in range(len(graph)):
        if graph[v][i] and not visited[i]:
            dfs(i, visited, graph)

n = int(input("Enter number of vertices: "))
graph = []

print("Enter adjacency matrix:")
for _ in range(n):
    graph.append(list(map(int, input().split())))

start = int(input("Enter starting vertex: "))
```

```
visited = [False]*n
print("DFS Traversal:")
dfs(start, visited, graph)
```

Sample Output

```
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
1 0 0 1
1 0 0 1
0 1 1 0
Enter starting vertex: 0
DFS Traversal:
0 1 3 2
```

Exercise No: 13

Dijkstra's Algorithm

Aim

To write a Python program to find shortest paths from a source vertex using Dijkstra's Algorithm.

Algorithm

Step 1: Start
Step 2: Read number of vertices n and cost matrix
Step 3: Initialize distance array with infinity and visited array with False
Step 4: Set distance of source vertex = 0
Step 5: Repeat n times:
    a) Select unvisited vertex with minimum distance
    b) Mark it visited
    c) Update distances of its adjacent vertices
Step 6: Display shortest distances
Step 7: Stop

Source Code

```
import sys


n = int(input("Enter number of vertices: "))

graph = []


print("Enter cost matrix:")

for _ in range(n):

    graph.append(list(map(int, input().split())))


source = int(input("Enter source vertex: "))

dist = [sys.maxsize]*n

visited = [False]*n

dist[source] = 0
```

```
for _ in range(n):
    u = -1
    min_dist = sys.maxsize
    for i in range(n):
        if not visited[i] and dist[i] < min_dist:
            min_dist = dist[i]
            u = i
    visited[u] = True
    for v in range(n):
        if graph[u][v] and not visited[v]:
            if dist[v] > dist[u] + graph[u][v]:
                dist[v] = dist[u] + graph[u][v]

print("Shortest distances from vertex", source, ":")
for i in range(n):
    print("Vertex", i, "Distance", dist[i])
```

Sample Output

Enter number of vertices: 3

Enter cost matrix:

0 1 4

1 0 2

4 2 0

Enter source vertex: 0

Shortest distances from vertex 0 :

Vertex 0 Distance 0

Vertex 1 Distance 1

Vertex 2 Distance 3

Exercise No: 14

Knuth-Morris-Pratt (KMP) String Matching

Aim

To write a Python program to search for a pattern in a text using KMP Algorithm.

Algorithm

Step 1: Start
Step 2: Read text T and pattern P
Step 3: Compute longest prefix-suffix (LPS) array for P
Step 4: Traverse text and compare characters using LPS to skip unnecessary comparisons
Step 5: If pattern found, display index
Step 6: Stop

Source Code

```
def compute_lps(pattern):
    lps = [0]*len(pattern)
    length = 0
    i = 1
    while i < len(pattern):
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1
        else:
            if length != 0:
                length = lps[length-1]
            else:
                lps[i] = 0
                i += 1
    return lps
```

```
text = input("Enter text: ")
pattern = input("Enter pattern: ")

lps = compute_lps(pattern)
i = j = 0

print("Pattern found at indices:")
while i < len(text):
    if pattern[j] == text[i]:
        i += 1
        j += 1
    if j == len(pattern):
        print(i-j, end=" ")
        j = lps[j-1]
    elif i < len(text) and pattern[j] != text[i]:
        if j != 0:
            j = lps[j-1]
        else:
            i += 1
```

Sample Output

Enter text: ABABDABACDABABCABAB

Enter pattern: ABABCABAB

Pattern found at indices:

10

Exercise No: 15

N-Queen Problem

Aim

To write a Python program to place N queens on an N×N chessboard such that no two queens attack each other using Backtracking.

Algorithm

Step 1: Start
Step 2: Place queens column by column
Step 3: For each column, try placing a queen in all rows
Step 4: If the position is safe, place the queen
Step 5: Recursively place queens in next column
Step 6: If conflict occurs, backtrack
Step 7: Display solution
Step 8: Stop

Source Code

```
def is_safe(board, row, col, n):
    for i in range(col):
        if board[row][i] == 1:
            return False
    i, j = row, col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1
    i, j = row, col
    while i < n and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
```

```
        j -= 1
    return True

def solve(board, col, n):
    if col >= n:
        return True
    for i in range(n):
        if is_safe(board, i, col, n):
            board[i][col] = 1
            if solve(board, col+1, n):
                return True
            board[i][col] = 0
    return False
n = int(input("Enter number of queens: "))
board = [[0]*n for _ in range(n)]
if solve(board, 0, n):
    print("Solution:")
    for row in board:
        print(row)
else:
    print("No solution")
```

Sample Output

Enter number of queens: 4

Solution:

[0, 0, 1, 0]

[1, 0, 0, 0]

[0, 0, 0, 1]

[0, 1, 0, 0]

Exercise No: 16

Hamiltonian Cycle

Aim

To write a Python program to find a Hamiltonian Cycle in a graph using Backtracking.

Algorithm

Step 1: Start
Step 2: Read adjacency matrix
Step 3: Initialize path with first vertex
Step 4: Recursively try adding vertices to path if safe
Step 5: If path includes all vertices and last vertex connects to first, cycle exists
Step 6: Display Hamiltonian cycle
Step 7: Stop

Source Code

```
V = 5  # Number of vertices


def isSafe(v, graph, path, pos):
    if graph[path[pos-1]][v] == 0:
        return False
    if v in path:
        return False
    return True


def hamCycleUtil(graph, path, pos):
    if pos == V:
        return graph[path[pos-1]][path[0]] == 1
    for v in range(1, V):
        if isSafe(v, graph, path, pos):
            path[pos] = v
```

```
            if hamCycleUtil(graph, path, pos+1):
                return True
            path[pos] = -1
    return False

def hamCycle(graph):
    path = [-1]*V
    path[0] = 0
    if not hamCycleUtil(graph, path, 1):
        print("Solution does not exist")
        return
    print("Hamiltonian Cycle:")
    print(path + [path[0]])

graph = [
    [0,1,0,1,0],
    [1,0,1,1,1],
    [0,1,0,0,1],
    [1,1,0,0,1],
    [0,1,1,1,0]
]

hamCycle(graph)
```

Sample Output

Hamiltonian Cycle:

[0, 1, 2, 4, 3, 0]

Exercise No: 17

Subset Sum Problem

Aim

To write a Python program to find all subsets of a set whose sum equals a given target using Backtracking.

Algorithm

Step 1: Start
Step 2: Read set of numbers and target sum
Step 3: Generate subsets recursively
Step 4: If subset sum equals target, display subset
Step 5: Continue until all subsets are checked
Step 6: Stop

Source Code

```
def subset_sum(nums, target, path=[]):
    if sum(path) == target:
        print(path)
        return
    if sum(path) > target:
        return
    for i in range(len(nums)):
        subset_sum(nums[i+1:], target, path + [nums[i]])
nums = [5,6,12,54,2,20,15]
target = 25
print("Subsets with sum =", target)
subset_sum(nums, target)
```

Sample Output

```
Subsets with sum = 25
[5, 6, 12, 2]
[5, 20]
```

Exercise No: 18

Travelling Salesman Problem (TSP)

Aim

To write a Python program to solve the Travelling Salesman Problem using Branch and Bound / Backtracking.

Algorithm

Step 1: Start
Step 2: Read number of cities n and cost matrix
Step 3: Start from first city
Step 4: Visit nearest unvisited city and mark as visited
Step 5: Repeat until all cities are visited
Step 6: Return to starting city
Step 7: Display minimum travelling cost
Step 8: Stop

Source Code

```
import sys


def tsp(graph, visited, curr, n, count, cost):
    if count == n and graph[curr][0]:
        return cost + graph[curr][0]
    ans = sys.maxsize
    for i in range(n):
        if not visited[i] and graph[curr][i]:
            visited[i] = True
            ans = min(ans, tsp(graph, visited, i, n, count+1, cost+graph[curr][i]))
            visited[i] = False
    return ans


n = int(input("Enter number of cities: "))
```

```
graph = []
print("Enter cost matrix:")
for _ in range(n):
    graph.append(list(map(int, input().split())))

visited = [False]*n
visited[0] = True

print("Minimum travelling cost =", tsp(graph, visited, 0, n, 1, 0))
```

Sample Output

Enter number of cities: 4

Enter cost matrix:

0 4 1 3

4 0 2 1

1 2 0 5

3 1 5 0

Minimum travelling cost = 7

Exercise No: 19

Approximation Algorithm – Vertex Cover

Aim

To write a Python program to find a Vertex Cover of a graph using a simple 2-approximation algorithm.

Algorithm

Step 1: Start
Step 2: Read number of vertices n and edges list
Step 3: Initialize empty vertex cover set
Step 4: While there are uncovered edges:
    a) Pick any edge (u, v)
    b) Add both u and v to vertex cover
    c) Remove all edges incident on u or v
Step 5: Display vertex cover set
Step 6: Stop

Source Code

```
n = int(input("Enter number of vertices: "))

e = int(input("Enter number of edges: "))


edges = []

print("Enter edges (u v):")

for _ in range(e):

    u, v = map(int, input().split())

    edges.append((u, v))


vertex_cover = set()


while edges:

    u, v = edges[0]

    vertex_cover.add(u)
```

```
    vertex_cover.add(v)
    edges = [edge for edge in edges if u not in edge and v not in edge]

print("Vertex Cover set:", vertex_cover)
```

Sample Output

Enter number of vertices: 4

Enter number of edges: 5

Enter edges (u v):

0 1

0 2

1 2

1 3

2 3

Vertex Cover set: {0, 1, 2, 3}

Exercise No: 20

Graph-Based Recommendation System (Simplified)

Aim

To write a Python program to suggest items to a user based on a graph of user-item interactions using adjacency list.

Algorithm

Step 1: Start
Step 2: Read number of users and items
Step 3: Create adjacency list representing which users interacted with which items
Step 4: For a given user, find items liked by similar users but not by the user
Step 5: Display recommended items
Step 6: Stop

Source Code

```
from collections import defaultdict


n_users = int(input("Enter number of users: "))

n_items = int(input("Enter number of items: "))


adj = defaultdict(set)

print("Enter user-item interactions (user item):")

for _ in range(int(input("Enter number of interactions: "))):

    u, i = map(int, input().split())

    adj[u].add(i)


user = int(input("Enter user for recommendation: "))

recommended = set()

for u, items in adj.items():

    if u != user:

        recommended.update(items - adj[user])
```

```
print("Recommended items for user", user, ":", recommended)
```

Sample Output

Enter number of users: 3

Enter number of items: 5

Enter number of interactions: 6

Enter user-item interactions (user item):

0 0

0 1

1 1

1 2

2 0

2 2

Enter user for recommendation: 0

Recommended items for user 0 : {2}

Exercise No: 21

Shortest Path Routing in a Network (Dijkstra's Algorithm)

Aim

To write a Python program to find shortest path routing in a network using Dijkstra's Algorithm.

Algorithm

Step 1: Start
Step 2: Read number of nodes n and cost matrix
Step 3: Read source node
Step 4: Initialize distances and visited array
Step 5: Repeat until all nodes visited:
    a) Select unvisited node with minimum distance
    b) Update distances of adjacent nodes
Step 6: Display shortest distances from source to all nodes
Step 7: Stop

Source Code

```
import sys

n = int(input("Enter number of nodes: "))

graph = []


print("Enter cost matrix:")

for _ in range(n):

   graph.append(list(map(int, input().split())))


source = int(input("Enter source node: "))

dist = [sys.maxsize]*n

visited = [False]*n

dist[source] = 0


for _ in range(n):
```

```
    u = -1
    min_dist = sys.maxsize
    for i in range(n):
        if not visited[i] and dist[i] < min_dist:
            min_dist = dist[i]
            u = i
    visited[u] = True
    for v in range(n):
        if graph[u][v] and not visited[v]:
            if dist[v] > dist[u] + graph[u][v]:
                dist[v] = dist[u] + graph[u][v]

print("Shortest distances from node", source, ":")
for i in range(n):
    print("Node", i, "Distance", dist[i])
```

Sample Output

Enter number of nodes: 4

Enter cost matrix:

0 1 4 0

1 0 2 1

4 2 0 5

0 1 5 0

Enter source node: 0

Shortest distances from node 0 :

Node 0 Distance 0

Node 1 Distance 1

Node 2 Distance 3

Node 3 Distance 2